Task Manager using bash
TA: youssef mehanna

Omar Wael 231007796
Ahmed Adel 231007728
Yassin Eslam 231017683

Table of Contents

1. Project Overview

The System Monitoring Dashboard is a comprehensive solution designed to monitor critical system metrics in real-time, providing both live visualization and historical reporting. The system is designed to automate system administration tasks by:

- Continuously monitoring CPU, GPU, memory, disk, network, and uptime metrics.
- Logging all data into structured directories for historical review.
- Streaming real-time updates to a frontend dashboard for visualization.
- Triggering alerts when any critical threshold is exceeded.

The system is built with the following technologies:

- Bash – Collects system metrics periodically.
- Node.js + Express – Provides REST API endpoints and WebSocket server for real-time data streaming.
- WebSocket– Streams live monitoring data to connected frontend clients.
- React + Recharts – Interactive frontend for visualization of metrics in graphs, charts, and dashboards.

The system is designed to be portable across Linux distributions and WSL (Windows Subsystem for Linux), with placeholders for macOS and Cygwin environments.

---

2. System Architecture

The architecture consists of three main layers:

1. Monitoring Layer (Bash Script)

- test.sh is the main Bash script that collects system metrics.
- It executes monitoring functions periodically.
- Data is logged into system_reports/YYYY-MM-DD-HHh-MMmin-SSsec.

---

2. Backend Layer (Node.js)

- Serves REST API endpoints for historical logs.
- Handles WebSocket connections for live monitoring.
- Parses log files into structured JSON using parseLogs.js.

---

3. Frontend Layer (React + Recharts)

- Displays real-time and historical system metrics.
- Uses interactive charts: line charts, circular progress bars, stacked bars.
- Responsive design with dark theme for readability.

---

3. Global Configuration & Thresholds

The Bash script defines critical thresholds to alert the user if hardware metrics exceed safe operating ranges:

Metric	**Critical Threshold**	**Action**
CPU Usage	> 70%	Alert logged and printed to console
CPU Temperature	> 50°C	Alert logged and printed to console
Memory (RAM) Usage	> 50%	Alert logged and printed to console
Virtual Memory (Swap)	> 50%	Alert logged and printed to console
GPU Usage	> 50%	Alert logged and printed to console
GPU Temperature	> 50°C	Alert logged and printed to console
Disk Usage	> 90%	Alert logged and printed to console

4. Core Monitoring Functions

The Bash script contains a set of specialized functions that collect and log system metrics:

CPU Monitoring

- Function: cpu()
- Description: Calculates total CPU usage using top.
- Temperature: Extracted using sensors (lm-sensors).
- Output: Logs time, usage %, and temperature (°C).

Example Log:

2025-12-17-18h-10min-00sec: CPU Usage: 25% CPU Temperature: 45°C

Memory Monitoring

- Function: memory()
- Description: Parses output of free.
- Metrics: Physical RAM usage, swap/virtual memory usage.
- Output: Total, used, and percentage utilization in GB.

Example Log:

2025-12-17-18h-10min-00sec: Memory Utilization: 40 Memory Used: 6 GB Memory Total: 16 GB

2025-12-17-18h-10min-00sec: Virtual Memory Utilization: 20 Virtual Memory Used: 3 GB Virtual Memory Total: 16 GB

GPU Monitoring

- Function: gpu()
- Supported Vendors: NVIDIA (nvidia-smi) and AMD (rocm-smi) and Intel (intel_gpu_top)
- Metrics: Usage %, temperature (°C).
- Validation: Checks if vendor-specific tool exists before running.

Example Log:

2025-12-17-18h-10min-00sec: GPU Usage: 30% GPU Temperature: 50°C

Disk Monitoring

- Function: disk()
- Tools Used: lsblk and df -h.
- Logic:
    - Lists physical disks and partitions.
    - Filters out system noise (zram, swap).
    - Logs mount points, used/total sizes, filesystem types.

Example Log:

2025-12-17-18h-10min-00sec: disk: sda 500G

2025-12-17-18h-10min-00sec: partition: sda1 250G ext4 / 100G used

---

Network Monitoring

- Function: network()
- Description:
    - Detects default network interface (ip route).
    - Reads raw byte counts from /sys/class/net/[interface]/statistics/.
    - Computes incoming/outgoing Bps by calculating deltas over time.

Example Log:

2025-12-17-18h-10min-00sec: Network Traffic | Interface: eth0 | Incoming_Bytes_Total: 1000000 | Outgoing_Bytes_Total: 500000

S.M.A.R.T. Disk Health Monitoring

- Function: smartStatus()
- Description: Uses smartctl to query health of disks.
- Metrics:
    - Overall SMART status (PASSED/FAILED)
    - Disk temperature
    - Reallocated sectors
    - Power-on hours

Example Log:

2025-12-17-18h-10min-00sec: SMART overall-health self-assessment test result: PASSED

Load Metrics & Uptime

- Function: loadmetrics()
- Description: Uses uptime to retrieve:
    - 1, 5, 15-minute load averages
    - Number of active users
    - System uptime

Example Log:

2025-12-17-18h-10min-00sec: 18:10:00 up 2:10, 1 user, load average: 0.25, 0.20, 0.15

---

5. Bash Script Lifecycle & OS Detection

1. OS Detection

- Function: detect_os()
- Checks:
    - Native Linux: executes main monitoring loop.
    - WSL: prints Windows warning and continues.
    - macOS/Cygwin: placeholder functions for cross-platform support.

2. Main Loop (linux())

- Description:
    1. Runs all monitoring functions in a loop.
    2. Interval is configurable (INTERVAL=10).

3. Sets trap for SIGINT for graceful exit.

- Workflow:
    1. Collect CPU, GPU, memory, disk, network, SMART, and uptime metrics.
    2. Write logs to structured folder.
    3. Sleep for interval. Repeat.

---

6. Utility & Validation Functions

assert_not_empty()

- Purpose: Guard function to prevent invalid logs.
- Functionality:
    1. Removes non-numeric characters using sed.
    2. Checks if output is empty or zero.
    3. Ensures data is valid before logging.

---

7. Unit Testing

The System Monitoring Dashboard includes a robust unit testing framework to ensure that all monitoring functions operate reliably and accurately. Unit tests validate each core component, detect anomalies, and confirm that collected data is meaningful before being logged or displayed on the frontend dashboard.

Purpose of Unit Testing

Unit testing provides several key benefits:

- **Accuracy:** Ensures that CPU, GPU, memory, disk, network, SMART, and load metrics are collected correctly.
- **Reliability:** Confirms that scripts execute without errors, even under unusual system conditions.
- **Early Detection of Issues:** Alerts administrators to invalid outputs, missing system tools, or unexpected values.
- **Consistency:** Guarantees that logs and real-time streams are formatted correctly for backend parsing and frontend visualization.

Testing Strategy

Unit tests are designed to mirror each monitoring function:

Component	Test Function	Purpose
CPU	test_cpu	Validates CPU usage and temperature values are numeric and within reasonable ranges.
Memory	test_memory	Checks physical RAM and swap usage percentages, ensuring values are accurate and consistent.
GPU	test_gpu	Confirms GPU usage and temperature metrics when supported vendor tools are available.
Disk	test_disk	Verifies that disks and partitions are properly detected, usage percentages are valid, and logs contain meaningful information.
Network	test_network	Ensures network interfaces and traffic statistics are detected and numerical values are consistent.
Load Metrics	test_loadmetrics	Validates system uptime and 1, 5, 15-minute load averages.

Component	Test Function	Purpose
SMART Status	test_smartStatus	Monitors disk health, temperature, reallocated sectors, and operational hours for validity.

---

8. Node.js Backend

Structure

server.js -> main entry point

routes/reports.js -> REST API

ws/monitor.js -> WebSocket handling

utils/parseLogs.js-> log parsing to JSON

middleware/ -> logger, error handler, 404 handler

REST API

Method	Route	Description
POST	/reports/folders	List of report folders (timestamps)
GET	/reports/folders/:folderName	Returns all logs in folder as JSON

WebSocket Protocol

- START → Launches Bash monitoring script via spawn.
- STOP → Terminates Bash script gracefully.
- Streams stdout and stderr from Bash to all connected clients.

Log Parsing Utilities

- CPU/GPU logs → JSON array with time, usage, temperature.
- Memory logs → JSON object with ram and virtual arrays.
- Disk logs → JSON array of snapshots, disks, and partitions.

- Network logs → JSON array with incomingBps, outgoingBps.
- Uptime logs → Load averages, users, uptime.
- SMART logs → Disk health status.

---

9. Frontend Dashboard

Charts

Component	Data Source	Description
CpuGpuChart	CPU & GPU logs	Line chart with usage and temperature
MemoryLineChart	RAM & Virtual logs	Line chart with usage %
MemoryCircular	RAM & Virtual logs	Circular progress bars
DiskLineChart	Disk snapshots	Line chart of disk usage (GB)
DiskProgressChart	Disk snapshots	Circular usage per disk/partition
DiskBarChart	Disk snapshots	Stacked bar chart of used/free space
LoadAverageChart	Uptime logs	Line chart for 1, 5, 15 min load averages
NetworkDashboard	Network logs	Charts for throughput and total bytes

UI Design

- Dark theme: #0a1f3c.
- Grid layout with spacing for readability.
- Legends and tooltips for all metrics.
- Responsive charts using Responsive Container.

10. GitHub and Docker Links

GitHub: https://github.com/AhmedAdelMohamedAbouhussein/bash_task_manager

Docker: https://hub.docker.com/repository/docker/ahmedadelabouhussein/os/general